

OPEN TECHNOLOGY COMPUTER EDUCATION

Advance C Programming Notes — Complete Master Manual

Course Instructor: Kanika Thakur

Chapter 1: Introduction to C Language

C is a powerful, general-purpose, structure-oriented programming language developed by Dennis Ritchie at AT&T's Bell Laboratories in 1972. It is widely known as the "mother language" of modern programming because many modern languages like C++, Java, and Python borrow syntax and core concepts from it.

Features of C Language

- **Simple and Efficient:** Clean syntax and fast execution speed.
- **Portable (Machine Independent):** C programs can run on different hardware platforms with little or no modification.
- **Mid-Level Language:** It bridges the gap between low-level (assembly) and high-level languages, allowing direct manipulation of hardware/memory using pointers.
- **Structured:** Programs can be divided into small, manageable modules using functions.

Chapter 2: Execution Flow and Structure of a C Program

Before writing code, it is important to understand how a C program is processed and built.

Compilation Process

A C program goes through four main phases to convert human-readable source code into machine-executable code:

1. **Preprocessing:** Processes lines starting with # (e.g., #include). It expands macros and includes header files.
2. **Compilation:** Converts the preprocessed source code into Assembly code.
3. **Assembly:** Converts Assembly code into Object code (.obj or .o).

4. **Linking:** Combines object files with standard library files to generate the final Executable file (.exe).

Structure of a C Program

```
// Section 1: Documentation (Comments)
// Program to display a welcome message

// Section 2: Link Section (Header files)
#include <stdio.h>

// Section 3: Global Declaration (Optional)

// Section 4: Main Function (Execution starts here)
int main() {
    // Statement Section
    printf("Welcome to Open Technology Computer Education!\n");
    return 0; // Return statement
}
```

Chapter 3: Tokens, Data Types, and Keywords

Tokens are the smallest individual units in a C program. A program is built by combining these tokens.

Types of Tokens

- **Keywords:** Reserved words that have a predefined meaning for the compiler. C has 32 standard keywords (e.g., int, return, if, const).
- **Identifiers:** Names given to program elements like variables, functions, and arrays.
- **Constants:** Fixed values that do not change during program execution.
- **Strings:** Sequences of characters enclosed in double quotes (e.g., "Kanika").
- **Operators:** Symbols used to perform operations (e.g., +, -, *, /).

Data Types in C

Data types define the type of data a variable can store and how much memory it occupies.

Data Type	Keyword	Size (Typical)	Format Specifier
Integer	int	2 or 4 Bytes	%d
Character	char	1 Byte	%c
Floating Point	float	4 Bytes	%f
Double Floating Point	double	8 Bytes	%lf

Chapter 4: Variables and Constants

Variable Declaration and Initialization

Declaration informs the compiler of a variable's data type and name, allocating memory. Initialization assigns an initial value to that allocated memory.

```
int systemTimeout;           // Declaration
systemTimeout = 3600;        // Initialization
float sensorDelta = 0.05f;    // Combined statement
```

Scope of Variables

Scope determines the visibility and accessibility of a variable across different regions of source code.

- **Local Variables:** Declared strictly inside a structural block or function execution boundary. They exist solely within that execution frame.
- **Global Variables:** Declared outside all functional bodies. They are visible throughout the entire compilation unit from the point of declaration until the termination of the host process execution thread.
- **Static Variables:** Declared using the static keyword. They retain their last mutated value across successive function calls, persisting in memory until the application terminates.

```
#include <stdio.h>

void executeCounter() {
    static int persistentCallCount = 0; // Initialized exactly once at
compile-time
    int dynamicLocalVar = 0;
    persistentCallCount++;
    dynamicLocalVar++;
    printf("Static Counter: %d, Local Counter: %d\n", persistentCallCount,
dynamicLocalVar);
}

int main() {
    executeCounter(); // Static: 1, Local: 1
    executeCounter(); // Static: 2, Local: 1
    return 0;
}
```

Constant Variables

Variables tagged with the `const` qualifier become immutable. Any attempt to reassign their value after declaration triggers a compilation error.

```
const double GRAVITY = 9.80665;
```

Chapter 5: Input and Output Functions

C has no built-in I/O primitives. Instead, it relies on standard library functions defined in `<stdio.h>`.

Formatted I/O Functions

- **printf():** Formats data and writes it to standard output.
- **scanf():** Reads formatted data from standard input, parsing data streams into locations specified by pointer arguments.

```
#include <stdio.h>

int main() {
    int userAge;
    printf("Enter your age: ");
    if (scanf("%d", &userAge) == 1) { // Validates successful extraction
        printf("Recorded Age: %d\n", userAge);
    }
    return 0;
}
```

Unformatted I/O Functions

- **getchar():** Reads the single next raw incoming character byte from standard input.
- **putchar(int ch):** Writes a single character byte to standard output.
- **fgets():** Safely reads a line of input by enforcing buffer limits (replaces the deprecated, unsafe gets() function).
- **puts(const char *str):** Writes a character string followed by an automatic trailing newline.

```
#include <stdio.h>

int main() {
    char inputBuffer[32];
    printf("Enter text: ");
    // Reads up to 31 chars safely, leaving room for '\0'
    fgets(inputBuffer, sizeof(inputBuffer), stdin);
    printf("Output via puts: ");
    puts(inputBuffer);
    return 0;
}
```

Chapter 6: Operators in C

- **Arithmetic Operators:** Perform mathematical operations: +, -, *, /, and % (Modulus-returns the integer remainder of division).
- **Relational Operators:** Evaluate equality or inequality, returning binary state representations: Boolean true (1) or false (0). Operators: ==, !=, <, >, <=, >=
- **Logical Operators:** Evaluate combinations of relational expressions. && (Logical AND) supports short-circuiting; || (Logical OR) checks alternative validity; ! (Logical NOT) inverts logic state.

- **Assignment Operators:** Assign values to variables, often combined with arithmetic shorthand: =, +=, -=, *=, /=, %=.
- **Increment/Decrement Operators:** Prefix (++x, --x) alters value before statement evaluation; Postfix (x++, x--) alters value after evaluation.

```
int baseValue = 5;
int prefixResult = ++baseValue; // baseValue becomes 6; prefixResult is 6
int postfixResult = baseValue++; // postfixResult takes 6; baseValue becomes 7
```

- **Bitwise Operators:** Manipulate integers at the binary level: & (AND), | (OR), ^ (XOR), ~ (NOT), << (Left Shift), >> (Right Shift).
- **Conditional Operator (Ternary):** A shorthand inline alternative to if-else. Syntax:

```
expression1 ? expression2 : expression3;
```

- **sizeof Operator:** A compile-time unary operator that returns the memory size, in bytes, of an expression, variable, or data type.

Chapter 7: Decision Making Statements

Decision-making blocks direct program execution based on runtime conditions.

if, if-else, and else-if Ladder

```
#include <stdio.h>

int main() {
    int testScore = 85;
    if (testScore >= 90) {
        printf("Grade: A\n");
    } else if (testScore >= 80) {
        printf("Grade: B\n"); // Executes and skips remaining ladder steps
    } else {
        printf("Grade: F\n");
    }
    return 0;
}
```

switch Case

An alternative to multi-way branch selection. It matches an integer or character expression against predefined constant labels.

```
#include <stdio.h>

int main() {
    char executionMode = 'V';
    switch (executionMode) {
        case 'D':
            printf("Diagnostic Mode Activated\n");
            break; // Prevents fall-through to the next case
        case 'V':
            printf("Verbose Logging Activated\n");
            break;
        default:
            printf("Standard Run Mode Activated\n");
    }
    return 0;
}
```

Chapter 8: Looping Statements

Loops repeat a block of code while a specific condition remains true.

while Loop

An entry-controlled loop that evaluates its test expression before executing the loop body.

```
int loopIndex = 0;
while (loopIndex < 5) {
    printf("%d ", loopIndex);
    loopIndex++;
}
```

do-while Loop

An exit-controlled loop that executes its body at least once before evaluating the test condition at the bottom.

```
int safeCounter = 10;
do {
    printf("Executed at least once");
} while (safeCounter < 5);
```

for Loop

A structured loop pattern that groups initialization, condition testing, and iteration updates into a single line.

```
for (int iterationStep = 0; iterationStep < 5; iterationStep++) {
    printf("%d ", iterationStep);
}
```

Nested Loops

Loops embedded within other loops, commonly used for processing multi-dimensional data structures like matrices.

Chapter 9: Jump Statements

Jump statements alter program flow unconditionally by transferring control to another location.

- **break:** Terminates execution of the innermost loop or switch block, passing control outside that scope.
- **continue:** Skips the remaining statements in the current loop iteration and passes control to the loop's next update/evaluation step.
- **goto:** Transfers program execution unconditionally to a named local label within the same function scope. *Warning: Generally discouraged ("spaghetti code").*
- **return:** Terminates execution of the current function and returns a value or control back to the caller frame.

Chapter 10: Arrays

An array is a contiguous block of memory that stores a fixed-size collection of elements of the same data type.

One-Dimensional Array

```
int primeSequence[5] = {2, 3, 5, 7, 11}; // Accessed from index 0 to 4
```

Two-Dimensional and Multi-Dimensional Arrays

Conceptually structured as a table of rows and columns.

```
int dataMatrix[2][3] = {{1, 2, 3}, {4, 5, 6}};  
int hyperCube[3][4][5]; // Multi-dimensional mapping
```

Character Arrays (Strings Basics)

In C, a character array functions as a string if it ends with a null terminator character (").

```
char stringLiteral[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Chapter 11: Strings

A string is a null-terminated sequence of characters stored in a contiguous array.

```
char explicitString[] = "Core C Language"; // Automatically inserts '\0'
```

String Input/Output

Using `scanf("%s")` stops reading input at the first whitespace character. To read an entire line safely, use `fgets()`.

String Manipulation Functions (<string.h>)

- `strlen(str)`: Returns length excluding null terminator.
- `strcpy(dest, src)`: Copies source string content into destination target.
- `strcat(dest, src)`: Appends source string to the end of destination string.
- `strcmp(str1, str2)`: Compares strings lexicographically. Returns 0 if identical.

```
#include <stdio.h>
#include <string.h>

int main() {
    char sourceText[20] = "Compilation";
    char destinationTarget[20];
    strcpy(destinationTarget, sourceText);
    printf("Length: %lu\n", strlen(destinationTarget)); // Output: 11
    return 0;
}
```

Chapter 12: Functions

Functions are self-contained, reusable blocks of code designed to perform a specific task, helping modularize large applications.

Function Mechanics

- **Function Prototype:** `int calculateSum(int parameterA, int parameterB);`
- **Call by Value:** Passes a copy of the argument; mutations do not affect the original variables.
- **Call by Reference:** Passes actual memory addresses using pointers, allowing direct modifications.

```
#include <stdio.h>

void modifyByReference(int *targetPointer) {
    *targetPointer = 999; // Modifies original value directly
}

int main() {
    int originalValue = 100;
    modifyByReference(&originalValue);
    printf("Value after function call: %d\n", originalValue); // Output: 999
    return 0;
}
```

Recursive Functions

A function that calls itself to solve a problem. It requires a well-defined base case to prevent infinite recursion and stack overflow errors.

Chapter 13: Storage Classes

Storage classes define the scope, visibility, lifetime, and initial default value of variables.

Storage Class	Keyword	Location	Default Value	Lifetime / Scope
Automatic	auto	CPU Stack Memory	Garbage Value	Block scope / Local
Register	register	CPU Registers	Garbage Value	Block scope / Local
Static	static	Data Segment RAM	Zero (0)	Entire program runtime / Local
External	extern	Data Segment RAM	Zero (0)	Entire program runtime / Global

Chapter 14: Pointers

A pointer is a variable that stores the memory address of another variable as its value.

```
int targetsTarget = 45;
int *addressPointer = &targetsTarget; // & returns address
*addressPointer = 90;                  // Dereferencing updates variable value
to 90
```

Pointer Arithmetic and Array Interaction

Pointers scale their arithmetic operations based on data type sizing. Incrementing an integer pointer advances it by 4 bytes (on standard systems). The array name serves as a constant pointer to its first element.

```
int elementArray[3] = {10, 20, 30};
int *arrayPtr = elementArray;
printf("%d", *(arrayPtr + 1)); // Accesses elementArray[1], printing 20
```

Special Pointer Categories

- **Void Pointer (void *):** A generic pointer type. Must be explicitly typecast before dereferencing.

- **Null Pointer:** Explicitly assigned NULL or 0 to indicate it does not point to a valid location.
- **Dangling Pointer:** Points to a memory location that has already been deallocated or freed.
- **Wild Pointer:** Declared but uninitialized; contains an unpredictable junk address.

Chapter 15: Structures

A structure is a user-defined data type that allows grouping variables of different data types under a single name.

```
struct SystemUser {
    int userID;
    char userName[50];
    float accountBalance;
};

// Initialization and Access
struct SystemUser adminUser = {1001, "Alice", 450.75f};
struct SystemUser technicalStaff[20]; // Array of structures
```

Members are accessed using the dot operator (.) for standard variables, or the arrow operator (->) when handling structural pointers.

Chapter 16: Unions

A union is a user-defined data type similar to a structure, but with one key difference: all its members share the exact same memory location. The total memory allocated is equal to the size of its largest member.

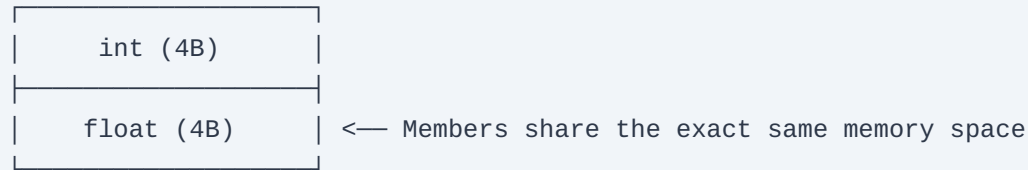
```
union SharedData {
    int integerValue;
    float floatingValue;
    char characterValue;
};
```

Memory Layout Comparison Layouts

Structure Memory Layout (Total = Sum of parts)

int (4B)	float (4B)
----------	------------

Union Memory Layout (Total = Size of largest member)



Chapter 17: Enumeration (enum)

An enumeration is a user-defined data type used to assign human-readable names to integer constants, making code cleaner and more maintainable.

```
enum NetworkState { STATE_DISCONNECTED, STATE_CONNECTING, STATE_CONNECTED };
```

By default, values start from 0 (0, 1, 2), but they can be explicitly overridden during definition if necessary.

Chapter 18: Dynamic Memory Allocation

Dynamic memory allocation allows a program to request and release memory from the system heap at runtime, rather than relying on fixed compile-time stack allocations.

- `malloc(size)`: Allocates raw uninitialized memory block containing junk data.
- `calloc(num, size)`: Allocates memory for an array and initializes all bytes to zero.
- `realloc(ptr, new_size)`: Resizes previously allocated heap space dynamically.
- `free(ptr)`: Releases memory back to the operating system to prevent memory leaks.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *dynamicArray = (int *)calloc(5, sizeof(int)); // Allocates 5 zeroed
integers
    if (dynamicArray == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    free(dynamicArray);
    dynamicArray = NULL; // Avoids dangling pointer risk
    return 0;
}

```

Chapter 19: File Handling

File handling allows a C program to store data permanently on a storage device by reading from and writing to files.

Core Processing Routine

```

#include <stdio.h>

int main() {
    FILE *fileStreamPointer = fopen("output.log", "w"); // Modes: "r", "w",
"a"
    if (fileStreamPointer != NULL) {
        fprintf(fileStreamPointer, "Log entry event occurred.\n");
        fclose(fileStreamPointer); // Flushes data stream to disk safely
    }
    return 0;
}

```

Chapter 20: Preprocessor Directives

The preprocessor processes source code before compilation begins, handling directives that start with a # symbol.

- `#include`: Imports contents of an explicit header file.

- **#define:** Creates symbolic constants or execution macro functions.

```
#define MAX_BUFFER_SIZE 1024
#define SQUARE_CALCULATION(val) ((val) * (val))
```

- **Conditional Compilation:** Controls which sections compile based on flags. Directives: `#ifdef`, `#ifndef`, `#if`, `#else`, `#endif`.

Chapter 21: Command Line Arguments

Command line arguments allow passing parameters to a C program from the terminal or command line when it is executed.

```
int main(int argc, char *argv[])
```

- **argc (Argument Count):** Total number of parameters entered (including application path execution name).
- **argv (Argument Vector):** An array of character strings. `argv[0]` points to the application path.

Chapter 22: Bitwise Programming

Bitwise operations manipulate individual bits of integer data types directly, which is useful for low-level optimizations and hardware programming.

- **& (Bitwise AND):** Sets bit to 1 if both compared inputs are 1.
- **| (Bitwise OR):** Sets bit to 1 if at least one input is 1.
- **^ (Bitwise XOR):** Sets bit to 1 if compared parameters differ.
- **<< (Left Shift):** Shifts bits left, padding with zeros; effectively multiplies by 2 per step.
- **>> (Right Shift):** Shifts bits right; effectively divides by 2 per step.

Chapter 23: Important C Programs

1. Prime Number Check

```
#include <stdio.h>

int isPrime(int n) {
    if (n <= 1) return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

int main() {
    int num = 29;
    printf("%d is %s\n", num, isPrime(num) ? "Prime" : "Not Prime");
    return 0;
}
```

2. Factorial of a Number (Using Recursion)

```
#include <stdio.h>

unsigned long long computeFactorial(int n) {
    if (n <= 1) return 1;
    return n * computeFactorial(n - 1);
}

int main() {
    printf("Factorial of 5: %llu\n", computeFactorial(5));
    return 0;
}
```


3. Fibonacci Series Generation

```
#include <stdio.h>

void printFibonacci(int limit) {
    int term1 = 0, term2 = 1, nextTerm;
    for (int i = 0; i < limit; i++) {
        printf("%d ", term1);
        nextTerm = term1 + term2;
        term1 = term2;
        term2 = nextTerm;
    }
    printf("\n");
}

int main() {
    printFibonacci(7); // Output: 0 1 1 2 3 5 8
    return 0;
}
```

4. Palindrome Number Verification

```
#include <stdio.h>

int checkPalindrome(int inputVal) {
    int processingVal = inputVal, remainderVal, reversedVal = 0;
    while (processingVal > 0) {
        remainderVal = processingVal % 10;
        reversedVal = (reversedVal * 10) + remainderVal;
        processingVal /= 10;
    }
    return (inputVal == reversedVal);
}

int main() {
    int num = 121;
    if (checkPalindrome(num)) {
        printf("%d is a Palindrome.\n", num);
    } else {
        printf("%d is not a Palindrome.\n", num);
    }
    return 0;
}
```

5. Armstrong Number Evaluation

```
#include <stdio.h>

int isArmstrong(int inputNumber) {
    int temporaryValue = inputNumber, remainderValue, calculatedSum = 0;
    while (temporaryValue > 0) {
        remainderValue = temporaryValue % 10;
        calculatedSum += (remainderValue * remainderValue * remainderValue);
        temporaryValue /= 10;
    }
    return (calculatedSum == inputNumber);
}

int main() {
    int num = 153;
    if (isArmstrong(num)) {
        printf("%d is an Armstrong number.\n", num);
    } else {
        printf("%d is not an Armstrong number.\n", num);
    }
    return 0;
}
```

6. Matrix Multiplication (Standard 2D Array)

```
#include <stdio.h>

void multiplyMatrices(int matA[2][2], int matB[2][2], int resultMat[2][2]) {
    for (int row = 0; row < 2; row++) {
        for (int col = 0; col < 2; col++) {
            resultMat[row][col] = 0;
            for (int k = 0; k < 2; k++) {
                resultMat[row][col] += matA[row][k] * matB[k][col];
            }
        }
    }
}

int main() {
    int matA[2][2] = {{1, 2}, {3, 4}};
    int matB[2][2] = {{5, 6}, {7, 8}};
    int result[2][2];
    multiplyMatrices(matA, matB, result);
    printf("Resultant Matrix:\n");
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            printf("%d ", result[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

7. String Reverse (In-Place Pointer Swap)

```
#include <stdio.h>
#include <string.h>

void reverseString(char *str) {
    int length = strlen(str);
    char *startPtr = str;
    char *endPtr = str + length - 1;
    char tempChar;
    while (startPtr < endPtr) {
        tempChar = *startPtr;
        *startPtr = *endPtr;
        *endPtr = tempChar;
        startPtr++;
        endPtr--;
    }
}

int main() {
    char text[] = "OpenTechnology";
    reverseString(text);
    printf("Reversed String: %s\n", text);
    return 0;
}
```

8. Bubble Sort Algorithm

```
#include <stdio.h>

void executeBubbleSort(int arrayData[], int elementCount) {
    int temporaryValue;
    for (int stepIndex = 0; stepIndex < elementCount - 1; stepIndex++) {
        for (int compareIndex = 0; compareIndex < elementCount - stepIndex - 1; compareIndex++) {
            if (arrayData[compareIndex] > arrayData[compareIndex + 1]) {
                temporaryValue = arrayData[compareIndex];
                arrayData[compareIndex] = arrayData[compareIndex + 1];
                arrayData[compareIndex + 1] = temporaryValue;
            }
        }
    }
}

int main() {
    int data[] = {64, 34, 25, 12, 22};
    int n = sizeof(data)/sizeof(data[0]);
    executeBubbleSort(data, n);
    printf("Sorted array: ");
    for(int i=0; i<n; i++) printf("%d ", data[i]);
    printf("\n");
    return 0;
}
```

9. Binary Search Algorithm

```
#include <stdio.h>

int performBinarySearch(int sortedArray[], int lowIndex, int highIndex, int
targetValue) {
    while (lowIndex <= highIndex) {
        int midIndex = lowIndex + (highIndex - lowIndex) / 2;
        if (sortedArray[midIndex] == targetValue) return midIndex;
        if (sortedArray[midIndex] < targetValue) lowIndex = midIndex + 1;
        else highIndex = midIndex - 1;
    }
    return -1;
}

int main() {
    int sortedData[] = {12, 22, 25, 34, 64};
    int n = sizeof(sortedData)/sizeof(sortedData[0]);
    int result = performBinarySearch(sortedData, 0, n - 1, 25);
    if(result != -1) printf("Element found at index: %d\n", result);
    else printf("Element not found\n");
    return 0;
}
```

Chapter 24: Interview Questions

C Theory Questions

Q1. Explain the difference between working declaration and working definition of a variable.

Answer: A declaration introduces a variable's name and data type to the compiler without allocating memory (e.g., using `extern int value;`). A definition tells the compiler what the variable is and allocates the actual memory space for it.

Q2. What is an activation record stack frame?

Answer: It is a dedicated block of memory allocated on the CPU stack whenever a function is called. It stores the function's local variables, parameters, return address, and execution state.

Output-Based Questions

Q1. Predict the output:

```
int x = 5;  
int y = x++ * ++x;  
printf("%d", y);
```

Answer: The behavior of this expression is **undefined**. Modifying a variable multiple times within the same sequence point (e.g., mixing `x++` and `++x` in a single statement) violates standard C rules and can produce different results on different compilers.

Pointer-Based Questions

Q1. What happens when you attempt to free a memory address pointer twice consecutively?

Answer: This causes a **Double Free Vulnerability**, which can corrupt the memory management structure of the heap and lead to undefined behavior or application crashes.

Viva Questions

Q1. Why is the size of an empty structure not zero in C?

Answer: C requires every unique object instance to have its own distinct memory address. Therefore, the compiler typically assigns a minimum size of 1 byte to an empty structure so it can be uniquely identified in memory.

Chapter 25: Practice Exercises

Beginner Level

1. Write a program to swap two variables without using a third temporary variable.
2. Accept a character input and determine whether it is a vowel or a consonant using a switch-case statement.
3. Print an isosceles star triangle pattern of size N using nested loops.

Intermediate Level

1. Implement a function that calculates the greatest common divisor (GCD) of two integers using the Euclidean algorithm.
2. Read an integer array and find both the largest and second-largest elements in a single pass.
3. Write a program to count the total number of vowels, consonants, digits, and spaces in a dynamically entered string.

Advanced Level

1. Build a custom system allocator using a fixed character array that mimics basic `malloc()` and `free()` functionality.
2. Write a program that reads a C source code file and strips out all single-line (`//`) and multi-line (`/* */`) comments.
3. Design a data structure using structures and bit-fields to pack a comprehensive calendar date (Day, Month, Year) into the smallest possible number of bytes.

Note for Students: Practice these programs in your lab sessions at Open Technology Computer Education. For any doubts regarding variables, loops, memory allocation, file handling, or bitwise operations, feel free to discuss them with Kanika Thakur in the upcoming lecture hours. All rights reserved.